T09 Pickups and Inventory

CRE132 worksheet

What you'll make

A bag for the player. Walk over coins, gems, keys and a potion: each one disappears and shows up in a row of icons at the top right of the screen, with how many of it you have.

On the way you'll meet the big idea of this sample: an **item** is not the object lying on the ground, it's the *kind* of thing. Each kind of item is a small data asset (a **ScriptableObject**, as in C09). The pickups, the bag and the UI all point at those assets, so a new kind of item needs no new code.

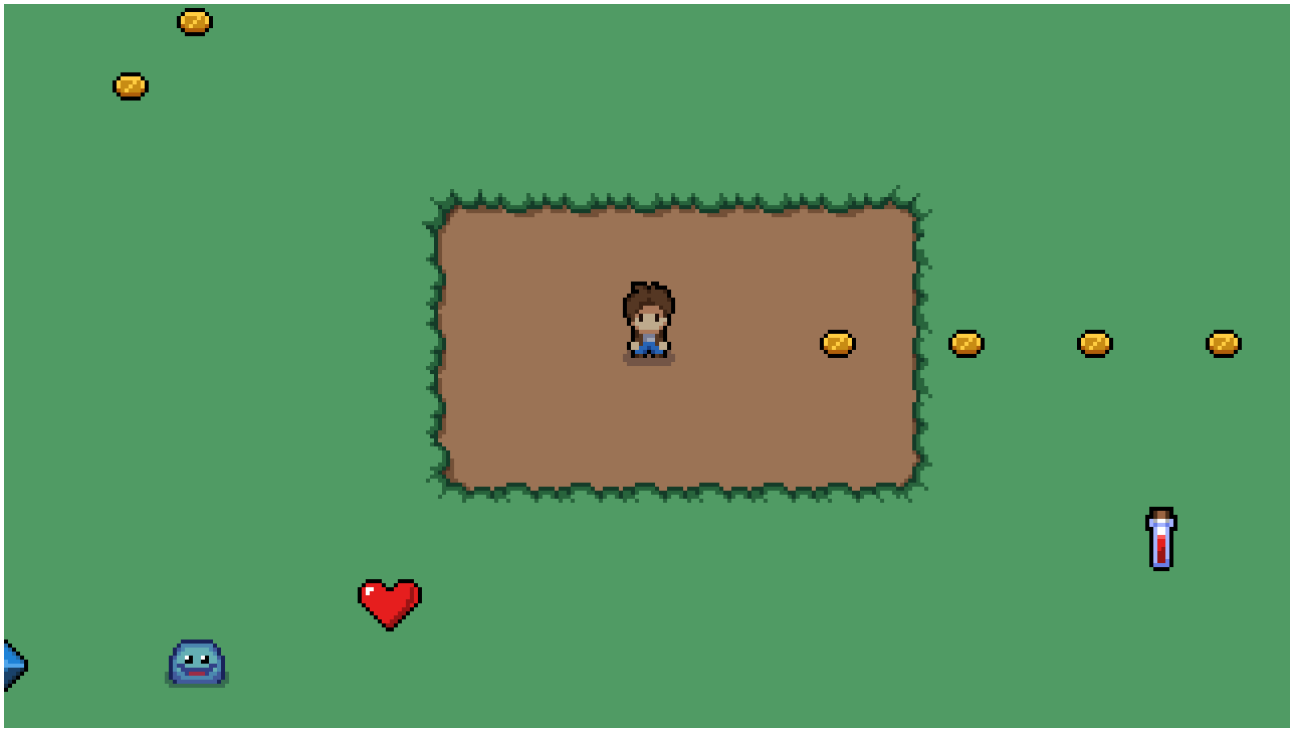


Figure 1: The finished scene: a trail of coins, gems, keys and a potion, with the inventory row at the top right.

Before you start

1. Unzip `CRE132_T09_PickupsAndInventory_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T09_PickupsAndInventory_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T09_PickupsAndInventory`.
3. Press **Play**. You can walk (WASD or arrows) and attack (J). The slimes, knockback and Game Over from T08 all work, and the **Hearts** already heal you (watch the health bar). The coins, gems, keys and potion do nothing yet.

The pickups are prefabs in `Assets/_game/Prefabs/Collectables/Items/`. The **HUD Canvas** has an empty object called **Inventory** at the top right, with a **Horizontal Layout Group** on it. You'll fill it with slots.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. What is an item? ItemData

Make a folder `Assets/_game/Scripts/T09_PickupsAndInventory` and create a script in it called `ItemData`. Replace everything with:

```
// A data asset for one kind of item: a coin, a key, a gem. Make one with
// Create > CRE132 > Item. Every coin pickup in the game points at the same Coin asset,
// and so does the inventory, so "is this a coin?" is just "is it the Coin asset?" (C09).
using UnityEngine;

[CreateAssetMenu(fileName = "New Item", menuName = "CRE132/Item")]
public class ItemData : ScriptableObject
{
    // Public, because this class is only a container: other scripts read these values.
    public string itemName = "Item";
    public Sprite icon;
    [TextArea] public string description;
}
```

- **`ScriptableObject`**, not `MonoBehaviour`: this script never goes on a GameObject. It describes a kind of asset that lives in the Project window, like a sprite or a sound.
- **`[CreateAssetMenu]`** adds **CRE132 > Item** to the **Create** menu, so you can make as many items as you like.
- **The fields are public** because ItemData is just a container of values for other scripts to read. **`[TextArea]`** gives the description a bigger box in the Inspector.

Add it: make a folder `Assets/_game/ScriptableObjects/Items`. Right-click in it and choose **Create > CRE132 > Item** four times. Name the assets and fill them in like this (the icons are in `Assets/_game/Art/Sprites/Kyrise/`):

- **Coin**: Item Name `Coin`, Icon `coin_01d`, Description `Shiny. Worth one.`
- **Blue Gem**: Item Name `Blue Gem`, Icon `gem_01c`, Description `A rare blue gem.`
- **Gold Key**: Item Name `Gold Key`, Icon `key_01d`, Description `Opens a gold door (T10).`
- **Potion**: Item Name `Potion`, Icon `potion_01a`, Description `Heals 2 (drinking it is a Try it).`

Nothing shows the descriptions yet: that's one of the Try it tasks.

2. The bag: Inventory

Create `Inventory` in the `T09_PickupsAndInventory` folder:

```
// The player's bag: a list of the items picked up. The same item can be in the list
// more than once (three coins are three entries), and other scripts ask how many of
// something there are.
using System.Collections.Generic;
using UnityEngine;

public class Inventory : MonoBehaviour
{
    // Serialized so you can watch the list fill up in the Inspector during Play.
    [SerializeField] private List<ItemData> items = new List<ItemData>();

    // Sent whenever the contents change, so the UI can redraw (a C# event, as in S09).
    public event System.Action Changed;
```

```
    // Other scripts can read the list; to change it they use Add and Remove.
    public List<ItemData> Items
    {
        get { return items; }
    }

    public void Add(ItemData item)
    {
        items.Add(item);
        Debug.Log("Picked up " + item.itemName + ", now " + CountOf(item));
        SendChanged();
    }

    // Takes one of the item out. Returns false if there wasn't one to take (T10 uses
    // this for keys).
    public bool Remove(ItemData item)
    {
        bool removed = items.Remove(item);
        if (removed)
        {
            SendChanged();
        }
        return removed;
    }

    public int CountOf(ItemData item)
    {
        int count = 0;
        foreach (ItemData heldItem in items)
        {
            if (heldItem == item)
            {
                count++;
            }
        }
        return count;
    }

    private void SendChanged()
    {
        // Nobody may be listening: an event with no subscribers is null.
        if (Changed != null)
        {
            Changed();
        }
    }
}
```

- **`List<ItemData>`** is a list that grows as you add to it. Three coins are three entries, all pointing at the same **Coin** asset.
- **`CountOf`** loops through the list and counts the entries that are the item you asked about. Comparing two items is just comparing two asset references with `==`.
- **`Remove` returns a `bool`**: `true` if there was one to take. T10's locked doors use this to spend a key.
- **`Changed` is an event** (S09): the Inventory doesn't know or care who's listening. The UI will listen and redraw.

Add it: select the **Player** and **Add Component > Inventory**. The **Items** list starts empty.

3. Things on the ground: ItemPickup

Create `ItemPickup` in the same folder:

```
// An item lying in the world. When the player touches it, it goes into their Inventory.
using UnityEngine;

public class ItemPickup : MonoBehaviour
{
    [SerializeField] private ItemData item;
    [SerializeField] private int amount = 1;

    private void OnTriggerEnter2D(Collider2D other)
    {
        // Only things with an Inventory (the player) can pick it up.
        Inventory inventory = other.GetComponent<Inventory>();
        if (inventory == null)
        {
            return;
        }

        for (int i = 0; i < amount; i++)
        {
            inventory.Add(item);
        }
        Destroy(gameObject);
    }

    // Runs in the editor whenever a value changes in the Inspector. Showing the item's
    // icon means a pickup always looks like the item it gives: change the item, and the
    // picture follows.
    private void OnValidate()
    {
        SpriteRenderer spriteRenderer = GetComponent<SpriteRenderer>();
        if (item != null && spriteRenderer != null)
        {
            spriteRenderer.sprite = item.icon;
        }
    }
}
```

- **The same trigger pattern as S01's potion**, but it asks for an `Inventory` instead of a `Health`. Slimes have no Inventory, so they walk over coins without picking them up.
- **The `for` loop** adds the item `amount` times, so one pickup could be a bag of five coins.
- **`OnValidate`** runs in the editor, not in the game, every time you change a value in the Inspector. It copies the item's icon onto the sprite, so the pickup always looks like what it gives.

Add it: this is where data pays off. Open each prefab in `Assets/_game/Prefabs/Collectables/Items/` (double-click it), add **ItemPickup** and drag the matching asset into the **Item** slot. Leave **Amount** at 1.

Prefab	Item
Coin Pickup	Coin
Blue Gem Pickup	Blue Gem
Gold Key Pickup	Gold Key
Potion Pickup	Potion

Leave the **Heart** prefab alone: hearts heal you straight away, so they use S01's **HealthPickup** (with **Player Only** ticked), not ItemPickup. They never go in the bag.

All four prefabs use the same script with a different asset. Because the pickups in the scene are copies of these prefabs, they all update at once.

Test it: press Play, select the **Player** and watch its **Inventory** in the Inspector while you walk over coins. The **Items** list grows, and the Console says `Picked up Coin, now 3`. Kill a slime: sometimes it drops a coin or a heart (T08's **DeathEffect**), and the coin goes in the bag too.

4. Showing the bag: the slot prefab and InventoryUI

First make one **slot**: an icon with a number in its corner.

1. In the Hierarchy, right-click the **HUD Canvas** and choose **UI > Image**. Rename it **Inventory Slot**. Set **Width** and **Height** to **80** and tick **Preserve Aspect**.
2. Right-click **Inventory Slot** and choose **UI > Text - TextMeshPro**. Rename it **Count**. In its Rect Transform set the **Anchors Min** and **Max** both to X **1**, Y **0** (the bottom-right corner), then **Pos X 4**, **Pos Y −6**, **Width 60**, **Height 36**.
3. On **Count**, set the text to `1`, **Font Size 30**, colour white, and the alignment buttons to **Right** and **Bottom**.
4. Drag **Inventory Slot** from the Hierarchy into `Assets/_game/Prefabs/UI/` to make a prefab, then delete it from the Hierarchy.

Now create `InventoryUI` in the `T09_PickupsAndInventory` folder:

```
// Shows the inventory as a row of slots: each kind of item once, with how many there are.
// It redraws only when the inventory changes (its Changed event), not every frame.
// Put it on a UI object with a Horizontal Layout Group, which lines the slots up.
using System.Collections.Generic;
using TMPro;
using UnityEngine;
using UnityEngine.UI;

public class InventoryUI : MonoBehaviour
{
    [SerializeField] private Inventory inventory;
    // A slot prefab: an Image for the icon, with a TMP_Text child for the count.
    [SerializeField] private GameObject slotPrefab;

    private void OnEnable()
    {
        inventory.Changed += Redraw;
    }

    private void OnDisable()
    {
        // The player (and its inventory) may already be gone, for example after dying.
        if (inventory != null)
        {
            inventory.Changed -= Redraw;
        }
    }

    private void Start()
    {
        Redraw();
    }
```

```
    private void Redraw()
    {
        // Remove the old slots. Destroy happens at the end of the frame, so this loop is
        // safe.
        foreach (Transform oldSlot in transform)
        {
            Destroy(oldSlot.gameObject);
        }

        // One slot per kind of item: remember which items already have a slot and skip
        // them.
        List<ItemData> shown = new List<ItemData>();
        foreach (ItemData item in inventory.Items)
        {
            if (shown.Contains(item))
            {
                continue;
            }
            shown.Add(item);

            GameObject slot = Instantiate(slotPrefab, transform);
            slot.GetComponent<Image>().sprite = item.icon;
            slot.GetComponentInChildren<TMP_Text>().text =
                inventory.CountOf(item).ToString();
        }
    }
}
```

- **`+=` in `OnEnable`** subscribes `Redraw` to the inventory's `Changed` event, and **`-=` in `OnDisable`** unsubscribes. The UI redraws only when something changes, not every frame.
- **`Redraw` throws the old slots away and makes new ones.** That's simpler than working out which slot to update, and fast enough for a handful of icons.
- **The `shown` list** remembers which items already have a slot. Three coins are three entries in the bag, but only one slot, showing **3**.
- **`Instantiate(slotPrefab, transform)`** makes the slot a child of **Inventory**, and the **Horizontal Layout Group** lines the children up for you.

Add it: select **HUD Canvas > Inventory** and add **InventoryUI**. Drag the **Player** into the **Inventory** slot and your **Inventory Slot** prefab into **Slot Prefab**.

Test it: press Play and pick things up. A new icon appears for each kind of item, and its number goes up for each extra one.

Check your work

- ☐ Coins, gems, keys and the potion disappear when you walk over them and go in the **Player**'s **Items** list.
- ☐ The top right shows one icon per kind of item, with the right count.
- ☐ Hearts heal you but don't appear in the row.
- ☐ Slimes walk over coins without picking them up. Coins they drop go in your bag.
- ☐ Compare with `CRE132_T09_PickupsAndInventory_Finished` if something behaves differently.

Key points

- **Data lives in assets, behaviour lives in scripts.** What an item *is* (name, icon, description) is an ItemData asset; picking up and showing are scripts. A new kind of item is a new asset and a prefab, with no new code.

- **A list can hold the same thing several times**, and counting it is a simple loop with ==.
- **Redraw the UI when something changes** (an event), not every frame.

If something goes wrong

- **CRE132 > Item isn't in the Create menu:** ItemData has an error, or its class doesn't say `: ScriptableObject`. Fix the first red error in the Console.
- **NullReferenceException in ItemPickup when you touch a pickup:** that prefab's **Item** slot is empty.
- **NullReferenceException in InventoryUI:** its **Inventory** or **Slot Prefab** slot is empty, or the Player has no Inventory component.
- **Pickups give items but the row stays empty:** InventoryUI is on the wrong object, or the slot prefab has no **Image** on its root.
- **Every slot shows "1":** the **Count** text isn't a child of the slot, so `GetComponentInChildren` can't find it.
- **The pickup in the scene still looks wrong after you changed its Item:** you edited the scene copy, not the prefab. Open the prefab itself.

Try it

1. Make a coin bag: a copy of **Coin Pickup** with **Amount**
   5.
2. Make a new kind of item (a red gem, say) with its own asset and prefab, without writing any code.
3. Drink a potion with Q: in a new script on the Player, call `inventory.Remove(potion)`, and if it returns `true`, call `health.Heal(2)`.
4. Show an item's name and description when the mouse is over its slot.
5. Harder: keep the inventory between scenes, using S02's GameManager or S08's save file.